

RAPPORT DE PROJET PSTL – M1 STL

**“ Rapport Intermédiaire sur le Projet
d’Étude et de Réimplantation du Code
Source de Prince Of Persia (1989) ”**

Projet réalisé par
Côme Lance-Perlick
Benoît Bled
Jules Ragu

Projet encadré par
Antoine Miné



**SORBONNE
UNIVERSITÉ**

SOMMAIRE

SOMMAIRE	2
1. CONTEXTE DU PROJET	3
1.1 Le Jeu	3
1.2 Le Projet	3
2. CAHIER DES CHARGES	4
2.1 Objectifs	4
Analyse et compréhension du code source	4
Documentation	4
Réimplantation	4
Validation et tests	4
2.2 Exigences techniques	4
2.3 Livrables	4
3. TÂCHES RÉALISÉES	5
3.1 Choix Technologiques	5
Langage	5
Bibliothèque graphique	5
3.2 Étude Du Code Source	5
Les fichiers du jeu	5
Résumé des fonctions de chaque fichier exploré	8
Particularités du code	9
3.3 Réimplantation	11
Réimplantation exacte	11
3.4 Décodage Des Images	12
3.5 Décodage Des Niveaux	14
3.6 Affichage Statique	15
Architecture graphique et adaptation de la résolution	15
Topologie du niveau et masquage des données	15
Superposition des couches et illusion de profondeur	15
4. TÂCHES RESTANTES	17
4.1 Détail Des Objectifs Restants	17
Physique et contrôles	17
Affichage dynamique	17
Logique et événements	17
Documentation	17
4.2 Rétro-Planning Prévisionnel	18
5. Bibliographie	19
5.1 Apple II	19
5.2 Microprocesseur 6502	19
5.3 Merlin (assembleur)	19
5.4 Sources du projet	19

1.CONTEXTE DU PROJET

1.1 Le Jeu

Le jeu Prince Of Persia est développé à partir de 1985 par Jordan Mechner et publié en 1989 par l'éditeur Broderbund pour l'ordinateur personnel Apple II. Le titre rencontre un succès considérable et est rapidement porté sur de nombreuses plateformes 8 et 16 bits, qu'il s'agisse d'ordinateurs personnels ou de consoles de jeu. Ceci a marqué le point de départ d'une franchise vidéoludique durable, toujours active et actuellement détenue par le studio Ubisoft. Ce projet se focalise plus spécifiquement sur la version originale du jeu développée pour l'Apple II.

Cette première version a été entièrement écrite en assembleur pour le microprocesseur MOS 6502, qui équipe l'Apple II. Celle-ci représente un intérêt particulier dans le cadre de l'histoire de l'informatique et du jeu vidéo, car son code source a été rendu public par son auteur sur GitHub. Il ne s'agit donc pas d'un code reconstitué par désassemblage, mais bien les sources originales telles qu'elles ont été écrites à l'époque. On y retrouve ainsi les commentaires d'origine, noms de variables, fonctions et étiquettes qui sont généralement perdus durant un processus d'assemblage. Cette multitude de données rend le code nettement plus lisible et fidèle aux intentions de son concepteur.

Par ailleurs, l'auteur a également diffusé une documentation technique datant de la sortie du jeu, qui constitue une ressource précieuse pour faciliter notre analyse et compréhension du code. Il est également possible de consulter un travail d'étude du jeu amorcé dans un blog bien que, demeurant à ce jour incomplet. De plus, il est possible de lancer le jeu original via émulation sur diverses plateformes, notamment directement dans un navigateur web.

1.2 Le Projet

Ce projet s'inscrit dans le cadre de notre première année de Master informatique, spécialité Science et Technologie du Logiciel, à Sorbonne Université. Celui-ci constitue au-delà de l'intérêt historique, un terrain d'application concret pour des compétences en rétro-ingénierie et en programmation bas-niveau. L'objectif est d'étudier en profondeur le jeu Prince Of Persia et d'en proposer une réimplantation moderne.

Il s'agit du prolongement d'un projet réalisé durant l'année universitaire 2024-2025, qui avait permis de recréer en C un affichage partiel et monochrome des niveaux du jeu. Ainsi, le projet actuel vise à corriger d'éventuelles erreurs et de poursuivre le travail en traduisant de nouvelles fonctionnalités du code source original telles que les animations ou bien la possibilité de jouer au jeu. Nous pouvons nous appuyer pour cela sur le rapport et le code source (disponible sur GitHub) réalisés par le précédent groupe.

2. CAHIER DES CHARGES

2.1 Objectifs

Analyse et compréhension du code source

- Étudier l'organisation du code assembleur du jeu
- Identifier les principales structures (variables et routines globales)
- Comprendre les interactions avec l'architecture matérielle (exemple: les optimisations mémoire)

Documentation

- Produire une documentation détaillant le rôle des principales parties du code
- Mettre en correspondance les éléments du code assembleur avec leur rôle dans le jeu

Réimplantation

- Traduire certaines parties du code dans un langage moderne
- Conserver une correspondance entre le code original et le nouveau

Validation et tests

- Vérifier que la réimplantation produit un résultat équivalent à celui du jeu original
- Tester les fonctionnalités implémentées

2.2 Exigences techniques

- Compréhension du microprocesseur 6502
- Analyse du macro-assembleur Merlin utilisé pour écrire le code source
- Étude de l'architecture mémoire de l'Apple II (notamment la mémoire vidéo)

2.3 Livrables

- Une analyse documentée du code source du jeu original (éventuellement dans le rapport)
- Une implémentation moderne d'une partie du jeu
- Un programme exécutable
- Un rapport détaillé expliquant la démarche et les choix techniques réalisés

3. TÂCHES RÉALISÉES

3.1 Choix Technologiques

Langage

Dans la continuité du groupe précédent, nous avons choisi de réimplanter le jeu en langage C. Ce langage étant suffisamment proche du matériel pour permettre une compréhension fine des contraintes bas-niveau, tout en offrant une meilleure portabilité, lisibilité et productivité comparé à l'assembleur utilisé pour développer le jeu initial.

Bibliothèque graphique

Pour effectuer les opérations graphiques (affichage à l'écran et extraction des images), nous avons décidé d'utiliser la bibliothèque graphique SDL2 (Simple DirectMedia Layer). Celle-ci offre un certain confort d'utilisation pour l'affichage tout en restant assez bas-niveau, portable et permissive (par exemple, pour écriture directe sur un tampon d'image).

3.2 Étude Du Code Source

Les fichiers du jeu

Afin d'acquérir une vision plus globale du code source, nous avons parcouru chaque fichier afin de comprendre la structure du projet et le déroulement des opérations. S'agissant de notre première expérience d'analyse de code 6502, il a fallu un certain temps d'adaptation pour saisir l'utilité de chaque fichier ainsi que leur manière d'interagir entre eux. Nous avons classé les fichiers assembleur selon leur contenu principal :

- Les **fichiers de constantes** : Définissent des noms symboliques pour des valeurs numériques, notamment des index afin de récupérer du contenu dans des tableaux.

EQ.S, GAMEEQ.S, MOVEDATA.S, SEQDATA.S, SOUNDNAMES.S, BGDATA.S,
HRPARAMS.S

- Les **fichiers de déclaration de variables (dum / ds)** : Réserve de l'espace sans y mettre de valeur. Équivalent des déclarations de variables globales en C. L'allocation de mémoire est présente dans tous les fichiers, mais est le contenu principal des membres de cette catégorie. Les autres fichiers allouent surtout leurs variables locales en page zéro via "dum \$f0".

EQ.S, GAMEEQ.S

- Les **fichiers de données** : Octets ou mots écrits en dur dans le binaire. Équivalent des tableaux `const` en C. Ces fichiers sont divisés en deux parties : un index d'entrées qui permet un accès direct à un élément de la table, suivi des données du tableau.

`TABLES.S, HRTABLES.S, SEQTABLE.S, FRAMEDEF.S`

- Le **code** : Représente la majorité des fichiers et inclut entre autres toutes les routines.

`TOPCTRL.S, CTRL.S, COLL.S, CTRLSUBS.S, AUTO.S, MISC.S, MOVER.S, SUBS.S, FRAMEADV.S, GRAFIX.S, HIRES.S, SOUND.S, SPECIALK.S, GAMEBG.S, UNPACK.S`

- Les **fichiers système** : Chargent le code en mémoire ou indiquent la version du code. Ces fichiers sont spécifiques à l'Apple II et ne nécessitent pas d'être traduits.

`MASTER.S, BOOT.S, VERSION.S`

À l'exception des fichiers 'Code', ces catégories ne contiennent pas de routines complexes, et la traduction se fait alors directement car on y déclare seulement des variables ou on y définit des données. La partie la plus diversifiée et qui requiert le plus gros travail de traduction se retrouve donc dans les fichiers 'Code'.

Pour structurer notre traduction, nous formons d'autres groupes selon les aspects fonctionnels du jeu. En voici une liste avec une explication non-exhaustive de leur fonctionnalités :

Boucle principale :

La boucle principale `MainLoop` contenue dans le seul fichier `TOPCTRL.S` orchestre en boucle infinie l'exécution de tout le reste dans un ordre défini à chaque image. Cela commence par la lecture clavier, et poursuit sur la logique, l'affichage, le son / la musique et la transition de niveau. Toute la logique du jeu est gérée par `NextFrame` (déplacement, IA des ennemis, passage d'un écran à l'autre), la détection de collision et l'affichage résultant se fait avec `FrameAdv`. Le fichier gère aussi le démarrage (`START`, `RESTART`, `INITSYSTEM`), les cinématiques (`cutprincess`, `playcut`), et la détection de victoire / défaite.

Contrôle et physique du personnage :

Ce sont les fichiers qui concernent le mouvement, l'état et les collision du personnage. On retrouve parmi ces fichiers `CTRL.S` qui va avoir pour routine principale `GENCTRL` qui permet le contrôle du personnage en lisant les entrées du joystick, puis en examinant l'état du personnage et enfin en appelant l'action correspond à l'entrée (courir, sauter, s'accrocher, combattre...). On retrouve aussi `CTRLSUBS.S`, une bibliothèque contenant des routines utilisées par `CTRL.S`. Ces fichiers interagissent avec le Fichier Données `SEQTABLE.S` afin d'y récupérer l'animation qui correspond à l'action calculée dans `CTRLSUBS.S`. `SEQTABLE.S` et essentiellement le script des animations du jeu. Avec le code de `COLL.S` sont gérées les collisions du personnage et le fichier `SUBS.S` est un autre dictionnaire avec des routines

comme **GRAVITY**, **ADDFALL** ou **STARTKID** (parmi d'autres) qui sont importantes pour la physique du personnage.

Affichage :

L'affichage est géré dans les fichiers **FRAMEADV.S**, **GRAFIX.S** et **HIRES.S**. Ces fichiers sont très conséquents et nous parleront plus en détail du fonctionnement de l'affichage dans la suite de ce rapport. Essentiellement, **FRAMEADV.S** est responsable de choisir quelles images sont affichées à l'écran en ajoutant les éléments graphiques à une liste. **HIRES.S** contient toutes les routines permettant d'afficher les éléments au bon endroit. Et **GRAFIX.S** relie les deux autres fichiers en gérant la liste produite par **FRAMEADV.S** et en coordonnant leur affichage avec **HIRES.S**. Dans **BGDATA.S** sont contenus des données permettant de choisir correctement quelle version d'un élément graphique doit être affiché (en renvoyant un index qui pointe vers le fichier correspondant), où il doit être affiché, et quel masque est utilisé.

Intelligence artificielle des ennemis :

Le comportement des ennemis est géré dans un unique fichier **AUTO.S**. La routine principale du fichier **AUTOCTRL** appelle, selon l'ennemi, un des six programmes: **GuardProg** (gardes), **ShadowProg** (shadow), **SkelProg** (squelettes), **VizierProg** (vizir), **MouseProg** (souris) et **KidProg** (lorsque le joueur est contrôlé). Chacun de ces programmes simule le comportement d'un personnage non-joueur en manipulant directement les variables de joystick (**JSTKX**, **JSTKY**, **btn**, **c1rF**...) comme si un vrai joueur appuyait sur les touches. Le fichier contient également des tables avec des paramètres de combat (les ennemis ont différentes probabilités de frapper, de parer, d'avancer, etc.). Lorsque le joueur passe d'un écran à un autre, une logique de transition d'écran est effectuée dans ce fichier pour ajouter, transférer ou mettre à jour les ennemis (**transferguard**, **updateguard**, **ADDGUARD**). Certains ennemis utilisent aussi une routine **AUTOPLAYBACK** permettant de rejouer des séquences de commandes pré-enregistrées.

Audio :

L'audio est géré dans les fichiers encore non explorés : **SOUND.S**, **SOUNDNAMES.S**

Animations du jeu :

Les animations sont gérées par les fichiers **SUBS.S**, **GAMEBG.S** et **SEQTABLE.S** qui pour ce dernier contient une table de toutes les animations du jeu. Cette table chargée en position **\$3000** définit l'intégralité du comportement visuel et physique de chaque personnage, image par image. L'index en début de code donne l'adresse de chaque animation nommée. Puis la seconde partie du code contient les données de chaque animation. Chaque animation est une mini-routine de données qui se termine invariablement par **goto + adresse** pour boucler, revenir à **stand** (le positionnement de base du personnage), ou enchaîner sur une autre animation.

Résumé des fonctions de chaque fichier exploré

Fichiers	Rôle
TOPCTRL.S	Boucle principale du jeu.
EQ.S GAMEEQ.S	Fichiers déclarant des variables et définissant des constantes , ils regroupent un grand nombre de déclarations, ils sont inclus dans pratiquement tous les autres fichiers.
MOVEDATA.S SEQDATA.S SOUNDNAMES.S BGDATA.S	Fichiers de constantes , fichiers très courts instanciant des valeurs des paramètres et des indexes correspondant à des points d'entrées dans certains tableaux. Inclus dans de nombreux fichiers.
HIRESPARAM.S HRTABLES.S FRAMEDEF.S SEQTABLES.S TABLES.S	Fichiers de données constant et spécifique à certaines fonctionnalités. Les données sont stockées dans des tableaux, l'accès à des données spécifiques se fait en utilisant les index présents en tête de fichier.
AUTO.S COLL.S CTRL.S CTRLSUBS.S	Fichiers d'interaction , gérant les interactions avec les différents éléments du jeu et ou le joueur.
FRAMEADV.S GAMEBG.S GRAPHIX.S HIRES.S SOUNDS.S SUBS.S	Fichiers son et images , s'occupant de l'affichage du son et des animations. L'affichage est expliqué plus en détail dans la suite du rapport.
SPECIALK.S	Fichier des inputs , contenant des fonctions de contrôle des touches tapées au clavier pour jouer.
BOOT.S MASTER.S	Fichiers système , permettant de lancer le jeu et de récupérer les données du jeu.
VERSION.S	Ne contient pas de code et indique la version du code .

Tableau récapitulatif des rôles des fichiers du code source étudié

Particularités du code

Code auto-modifiant

Le code peut se modifier en cours de route afin d'optimiser de la mémoire et / ou des cycles d'exécution (éviter des sauts conditionnels ou des chargements de valeurs à chaque appel).

Par exemple, afin d'éviter de devoir utiliser une variable booléenne pour savoir si l'initialisation a déjà eu lieu, on désactive la routine d'initialisation. Cette désactivation consiste à placer au début de la fonction (remplacer l'octet après le label) une instruction de retour de routine (RTS codée `$60` en hexadécimal)

23	entry lda #\$60
24	sta entry

Extrait du fichier `BOOT.S`

Multiplication via "Look-Up Table" (LUT)

Le processeur MOS 6502 ne possède pas d'instruction de multiplication matérielle et la multiplication logicielle implique une boucle ou une combinaison d'instructions (multiplication possible par un décalage de 2 bits vers la gauche et un ajout de la valeur initiale 2 fois).

Cependant, c'est bien trop lent pour un jeu vidéo en temps réel. C'est la raison pour laquelle, la multiplication par 10 a été simplifiée en une lecture de valeur dans un tableau contenant pour chaque indice *i*, la valeur précalculée de $10 * i$.

71	lda Mult10,y
72	sta yindex ;block # (0-29)

Extrait du fichier `FRAMEADV.S`

Variables globales comme arguments et retours

Plutôt que de passer des paramètres à une fonction via la pile comme le fait le langage C, le jeu écrit les arguments dans des variables globales spécifiques avant d'appeler une routine, puis lit le résultat depuis d'autres variables globales. Ces variables globales sont situées dans la page zéro (256 premiers octets de la RAM dont les adresses occupent 1 seul octet) afin de bénéficier d'accès plus rapides (grâce à des instructions spécifiques).

Cette manière de faire est dû au fait que le processeur 6502 ne possède que 3 registres (A, X et Y), ce qui est insuffisant pour passer plusieurs paramètres et faire des opérations dessus. De plus, la manipulation de la pile est lente et limitée à 256 octets.

```

604 *-----
605 *
606 *   Get objids & states of 10 blocks in row below,
607 *   1 block to left
608 *
609 *   In:  rowno
610 *   Out: BELOW/SBELOW [0-9]
611 *
612 *   Use getbelow1 to look at screens other than scrnBelow
613 *   (In: A = scrn #)
614 *
615 *-----
616 getbelow

```

Extrait du fichier [FRAMEADV.S](#)

Déclaration différée

Le code utilise des variables globales qui ne sont définies nulle part dans le fichier courant, mais qui sont déclarées avec des adresses mémoire absolues.

Cela permet de partager la même adresse mémoire entre différents modules qui ne sont pas forcément en mémoire au même moment, ou d'associer des noms de variable à des adresses matérielles spécifiques (ex: mémoire vidéo de l'Apple I).

25	dum locals
26	index ds 1
27	rowno ds 1

Extrait du fichier [FRAMEADV.S](#)

Tables de sauts

L'utilisation de tableaux contenant des instructions de saut pointants vers différentes routines permet la déclaration différée des routines afin de les rendre accessibles depuis tout le programme.

7	org org
8	
9	jmp SURE
10	jmp FAST
11	jmp GETINITOBJ

Extrait du fichier [FRAMEADV.S](#)

3.3 Réimplantation

Réimplantation exacte

Lors de nos premiers essais, nous avons entrepris de traduire le chargeur de démarrage `BOOT.S`. Nous avons rapidement abandonné cette piste, car il n'est plus pertinent de réimplanter ce type de fichier aujourd'hui du fait que les architectures modernes n'ont plus besoin de manipulations mémoire ROM propres à l'Apple II. Cette tentative nous a néanmoins permis de nous familiariser avec la syntaxe et les optimisations du code.

Nous avons ensuite analysé le système d'affichage, structuré en 3 niveaux d'abstraction (étudié par le groupe précédent en détail et brièvement résumé dans la section affichage de l'étude du code). Nous avons réimplanté un extrait de `FRAMEADV.S` (le plus haut niveau) en traduisant le plus fidèlement possible le code d'origine pour en extraire des routines `SURE` et `FAST`. La comparaison avec le code de l'équipe précédente nous a confirmé que nos approches étaient très similaires. Réutiliser le code existant en y apportant quelques modifications nous paraît donc être une bonne idée pour gagner du temps et atteindre nos objectifs.

```
Dy = BlockBot[rowno + 1];  
Ay = Dy - 3;  
  
yindex = Mult10[rowno];  
PRECED = PREV[rowno];
```

```
Dy = BlockBot[rowno + 1];  
Ay = Dy - 3 - 1; // -1 with carry  
  
yindex = rowno * 10;  
  
PRECED = PREV[rowno]; // prev_object
```

Exemple de différences (fichiers `frameadv.c` avec le code du groupe précédent à gauche)

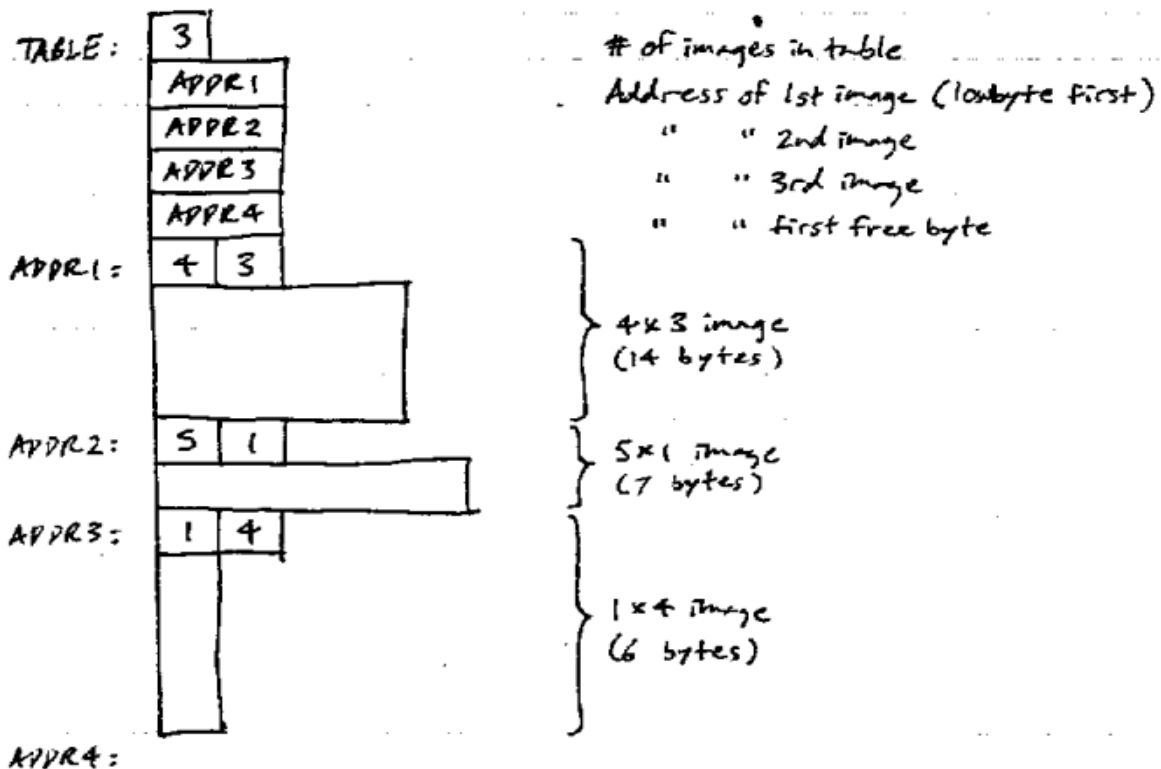
Sur cet exemple, nous pouvons voir comme différences que nous avons retiré également la retenue à Ay (car l'instruction `sec` pour activer la retenue est présente) et remplacé le tableau de multiplication précalculé par une multiplication directe.

3.4 Décodage Des Images

Le jeu charge les images en RAM à l'adresse \$6000. L'extraction de ces données nécessite de lire la structure binaire suivante :

Taille (en octet)	Position (en octet)	Description
1	0	N := nombre d'images dans le fichier
2 * N	2	Offsets := tableau des positions absolues ($\geq \$6000$)
?	2 * N + 1	Images := tableau de structures images, chaque image i se trouve en position $\text{Offsets}[i] - \$6000$

SAMPLE IMAGE TABLE



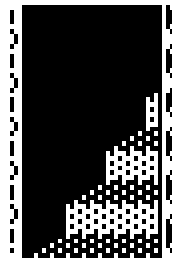
Note: Image data bytes are read off left-right, bottom-top.

Page 22 de [popsources.pdf](#) (documentation technique réalisée par l'auteur)

La structure d'une image se présente sous cette forme :

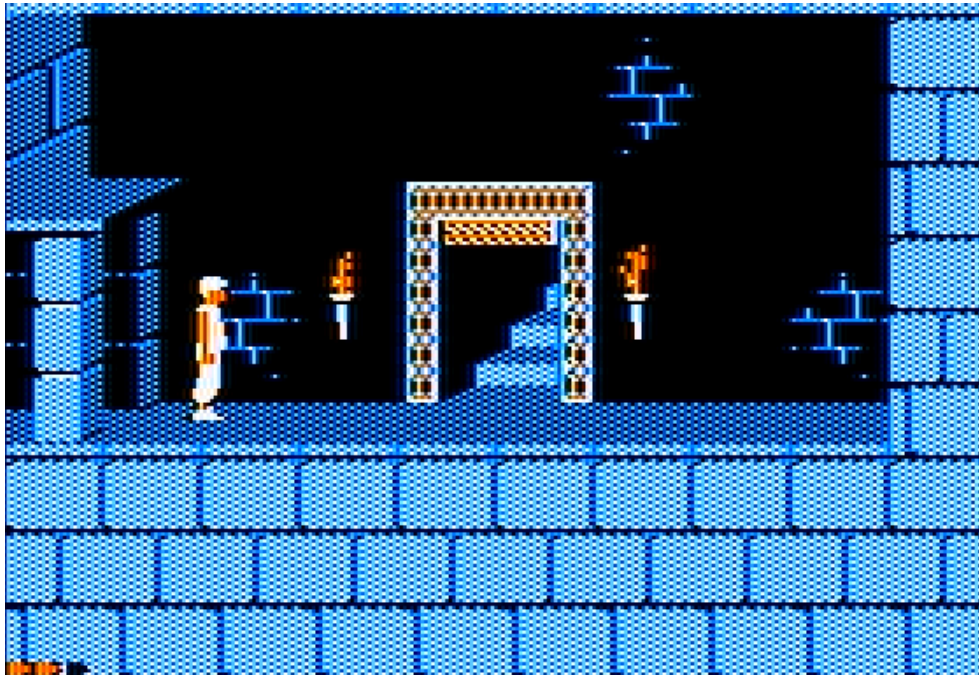
Taille (en octet)	Position (relative en octet)	Description
1	0	W := nombre d'octets par ligne
1	1	H := nombre de lignes
W * H	2	Données := chaque octet (à lire de gauche à droite et de bas en haut) contient 7 pixels qui peuvent être soit allumés soit éteints (le huitième indique la palette de couleur)

Le décodage de ce format a été implémenté dans le fichier `read_images.c`. Ce programme prend un fichier source et génère des images PNG dans le fichier `out/%nom/%n.png` avec `%nom` le nom du fichier d'entrée et `%n` le numéro de l'image. Ce code est une adaptation de celui fourni par l'équipe précédente avec les changements suivants : sortie au format PNG (réduit la taille de fichier) et fait le rendu sur des surfaces sans création de fenêtre graphique pour plus de performances.



Escalier (image 106 de `IMG.BGTAB1.PAL`)

3.5 Décodage Des Niveaux



Exemple de niveau ([source](#))

Chaque niveau contient 24 écrans, eux-mêmes composés de 30 blocs, sur 10 colonnes et 3 lignes. Chaque bloc est défini par un type (identifiant de l'objet) et un état logique (par exemple, une plaque de pression pressée). De plus, chaque écran peut être lié aux autres sur chaque direction (haut, bas, gauche et droite) à l'aide d'indices, permettant ainsi d'obtenir des formes différentes pour les niveaux.

La structure d'un niveau est la suivante :

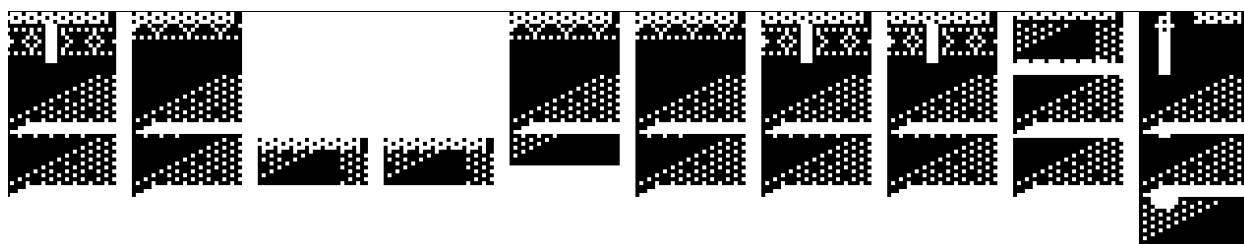
Taille (en octet)	Position (en octet)	Description
$24 * 30 = 720$	720	BlueType := identifiant des objets pour chaque écran
$24 * 30 = 720$	720	BlueSpec := état des objets pour chaque écran
256	1440	Link_loc := position des portes que l'on peut ouvrir avec un plaque de pression
256	1696	Link_map := information sur l'interaction avec les plaques de pression (timer, ...)
$24 * 4 = 96$	1952	Voisins := indices des écrans voisins pour chaque écran
64	2048	Informations réservées par l'éditeur

196	2112	Position des personnages dans le niveau
-----	------	---

3.6 Affichage Statique

Architecture graphique et adaptation de la résolution

Dans le but d'obtenir un premier rendu du jeu, nous avons mis en place une fenêtre graphique en utilisant la bibliothèque SDL2. La résolution originale de Prince of Persia étant de 280 par 192 pixels, un affichage direct sur des écrans modernes aurait produit une image bien trop petite. C'est pour cela que nous appliquons une mise à l'échelle (*4) à tous les éléments visibles, dont les images de fond extraites de [BGTAB1](#) et [BGTAB2](#).



Premier affichage dysfonctionnel

Topologie du niveau et masquage des données

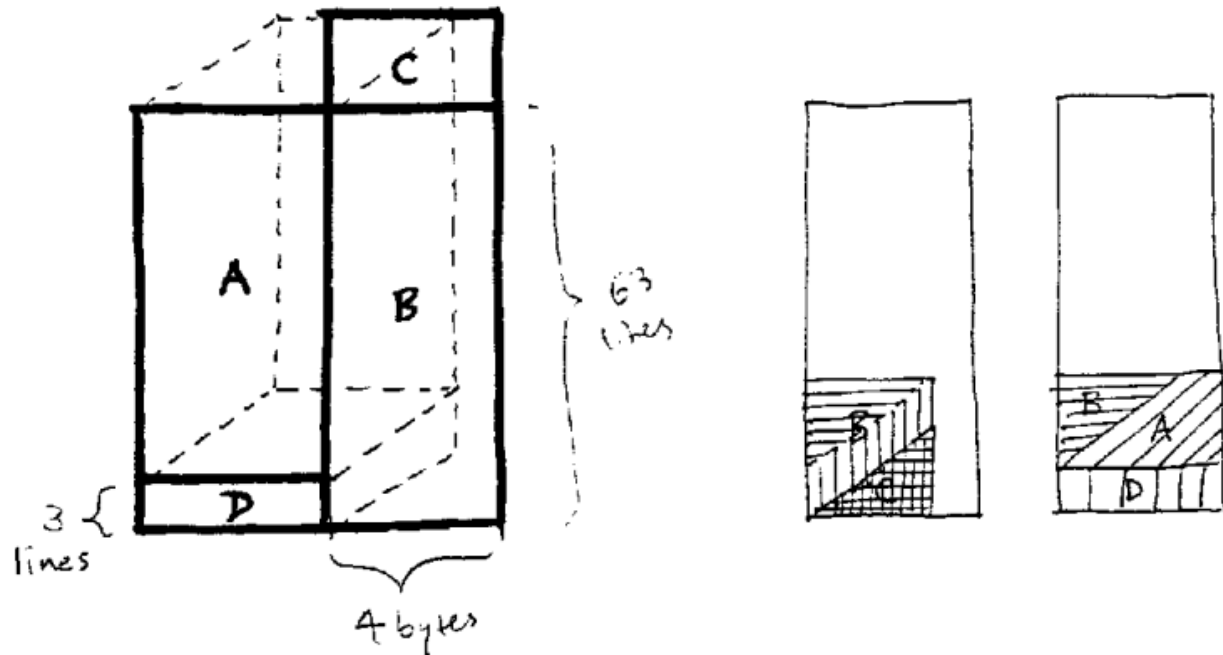
L'écran de jeu est découpé en une grille logique stricte de 10 colonnes par 3 lignes. Chaque bloc d'une dimension de 28 pixels de largeur sur 64 pixels de hauteur contient un identifiant auquel il faut appliquer l'opération logique `& $1F` afin de conserver uniquement les 5 bits de poids faible (les 3 bits de poids fort sont ignorés et restent encore un mystère). La valeur obtenue (entre 0 et 31) sert alors d'indice pour récupérer les portions d'images correspondantes dans les tables [piecea](#), [pieceb](#), [piecec](#) et [pieced](#) présentes dans le fichier [BGDATA.s](#) du code source original.

Superposition des couches et illusion de profondeur

Le génie technique de Prince of Persia sur Apple II réside dans son rendu 2.5D, créant une illusion de profondeur à partir d'éléments en 2D. Pour chaque bloc de la grille, le rendu visuel est décomposé en plusieurs couches distinctes. D'après l'étude de la documentation technique originale ([popsources.pdf](#)), nous avons identifié le rôle de ces couches :

- Couche C : Arrière plan haut, situé au-dessus de B.
- Couche B : Profondeur latérale, située à droite de A.
- Couche D : Le sol, situé juste en dessous de A.
- Couche A : La face avant de l'élément.
- Frontpiece : Le premier plan (ex: un pilier) derrière lequel le personnage peut passer.

Pour gérer correctement les chevauchements et respecter cette perspective, nous dessinons chaque élément du plus éloigné au plus proche. L'ordre d'affichage itératif pour un bloc donné est donc C, B, D, A puis Frontpiece.



4. TÂCHES RESTANTES

L'objectif principal avant la fin de l'année universitaire est d'obtenir une version minimale jouable du jeu, permettant au joueur de se déplacer entre les écrans. Étant donné l'ampleur du projet, la production d'une documentation technique reste une priorité absolue pour assurer la pérennité du travail pour d'éventuels prochains groupes.

4.1 Détail Des Objectifs Restants

Physique et contrôles

Afin de rendre le jeu un minimum jouable, il sera nécessaire d'étudier plusieurs composants du jeu tels que la physique, l'affichage du joueur et le calcul de la position du joueur dans le niveau. Nous pourrions également implémenter une fonctionnalité pour explorer les niveaux du jeu sans avoir besoin de lancer une partie, permettant une meilleure compréhension du level design et de repérer d'éventuels bugs graphiques.

Affichage dynamique

Il faudrait également rendre l'affichage plus dynamique avec l'implémentation des animations des éléments du décor et d'afficher les cinématiques du jeu. Il serait également intéressant d'implémenter l'affichage des couleurs du jeu pour un affichage plus proche des images du jeu.

Logique et événements

On peut également considérer l'étude du code de gestion automatique des ennemis et, dans une moindre mesure, l'étude du code des événements du jeu tels que les interactions avec les plaques de pression, la fin du niveau et des éléments contextuels comme l'épée du premier niveau.

Documentation

S'agissant d'un projet plutôt ambitieux, la partie la plus importante à réaliser est la documentation technique afin qu'un groupe passant après nous dispose de toutes les ressources nécessaires pour poursuivre le projet et le comprendre sans peine. Cela passe notamment par la documentation de chaque fichier et la création d'interfaces en C pour faciliter leur traduction. Nous disposons maintenant d'une bonne base mais il reste encore beaucoup à analyser. Avant le mi-semestre, certains fichiers (notamment `MISC.S`, `MOVER.S`, `SOUND.S` et `UNPACK.S`) n'ont pas encore pu être explorés en détail et seront étudiés pour la suite du projet.

4.2 Rétro-Planning Prévisionnel

Cette partie présente des estimations du temps d'implémentation des fonctionnalités du projet en considérant les projets des autres matières et en parallélisant certaines tâches.

Semaine 1 à 2 : Colorisation des images

Implémentation de la colorisation des images du jeu. Cette fonctionnalité étant relativement simple, elle peut être réalisée en moins de deux semaines.

Semaine 2 à 4 : Cinématiques

Implémentation des cinématiques du jeu, qui nécessitera probablement plus de deux semaines. Leur affichage en couleur sera assuré en lien avec la fonctionnalité de colorisation des images. Si la colorisation prend trop de temps, nous passerons à la prochaine étape afin d'explorer plus d'aspects du projet.

Semaine 2 à 5 : Animations du décor

Mise en place des animations des éléments du décor (torches, apparition des piques, etc.). Il s'agira d'abord d'animer les sprites individuels, puis de les ajouter dans l'affichage du décor. Le temps estimé pour cette étape est d'environ un mois car il ne pourra être finalisé qu'après l'implémentation des cinématiques du jeu. L'animation des personnages joueurs et ennemis peut aussi être envisagée lors de cette période.

Semaine 6 jusqu'à la fin : Gameplay (physique, déplacements, collisions)

Travail sur les mécaniques de gameplay, notamment la physique, les déplacements et les collisions. Cette partie du code source n'ayant pas encore été étudiée en détail, elle demandera davantage de temps. La durée est estimée entre un et deux mois.

En parallèle (sur environ 2 semaines cumulées) : Exploration des niveaux

L'implémentation de l'exploration des niveaux pourra être implémentée en moins de deux semaines en parallèle des autres fonctionnalités par un membre du groupe.

Tout au long du projet : Documentation

Rédaction de la documentation du projet, qui sera effectuée en continu en parallèle du développement et se poursuivra jusqu'à la fin du projet. Dans l'éventualité de la reprise du projet par un prochain groupe, nous voulons leur rendre la compréhension aussi facile que possible en leur mettant un maximum d'éléments à disposition.

5. Bibliographie

5.1 Apple II

- Apple II graphics. *Wikipedia*.
https://en.wikipedia.org/wiki/Apple_II_graphics
- Apple IIe Technical Reference Manual. Apple Computer, Inc., Addison-Wesley, 1985.
https://dn721808.ca.archive.org/0/items/Apple_IIe_Technical_Reference_Manual/Apple_IIe_Technical_Reference_Manual_text.pdf
- Apple II series timeline. *Wikipedia*.
https://en.wikipedia.org/wiki/Timeline_of_the_Apple_II_series

5.2 Microprocesseur 6502

- MOS Technology 6502 instruction tables. Masswerk.
<https://www.masswerk.at/6502/instruction-tables/>
- MOS Technology 6502 cheat sheet. Cheatsheets.one.
<https://cheatsheets.one/tech/6502>
- MOS Technology 6502 tutorial. Easy6502.
<https://skilldrick.github.io/easy6502/>

5.3 Merlin (assembleur)

- Merlin Macro Assembler. *Merlin – A Macro Assembler*. SDS, 1983.
<https://mirrors.apple2.org.za/ftp.apple.asimov.net/documentation/applications/misc/Merlin%20-%20A%20Macro%20Assembler%20%28SDS%2C%201983%29%20OCR.pdf>
- Merlin 816 documentation.
<http://www.apple-iigs.info/doc/fichiers/merlin816.pdf>

5.4 Sources du projet

- Prince of Persia source code (Apple II). GitHub.
<https://github.com/jmechner/Prince-of-Persia-Apple-II>
- Projet PSTL. GitHub.
<https://github.com/OrianeCrouzet/PSTL>